Get unlimited access Open in app

Published in Towards Data Science

This is your **last** free member-only story this month. Upgrade for unlimited access.

Mauro Di Pietro Follow
Jun 13, 2021 · 11 min read ★ · Listen

Save

Web Development with Python: Dash (complete tutorial)

Draw with Plotly, Embed Bootstrap CSS, Upload & Download files, Change Inputs after selection, Navbars, Spinners, and more...

Summary

Welcome to this hardcore Dash tutorial, following this article you will be able to produce and deploy a basic prototype (minimum viable product) for any kind of web application.

Get unlimited access Open in app

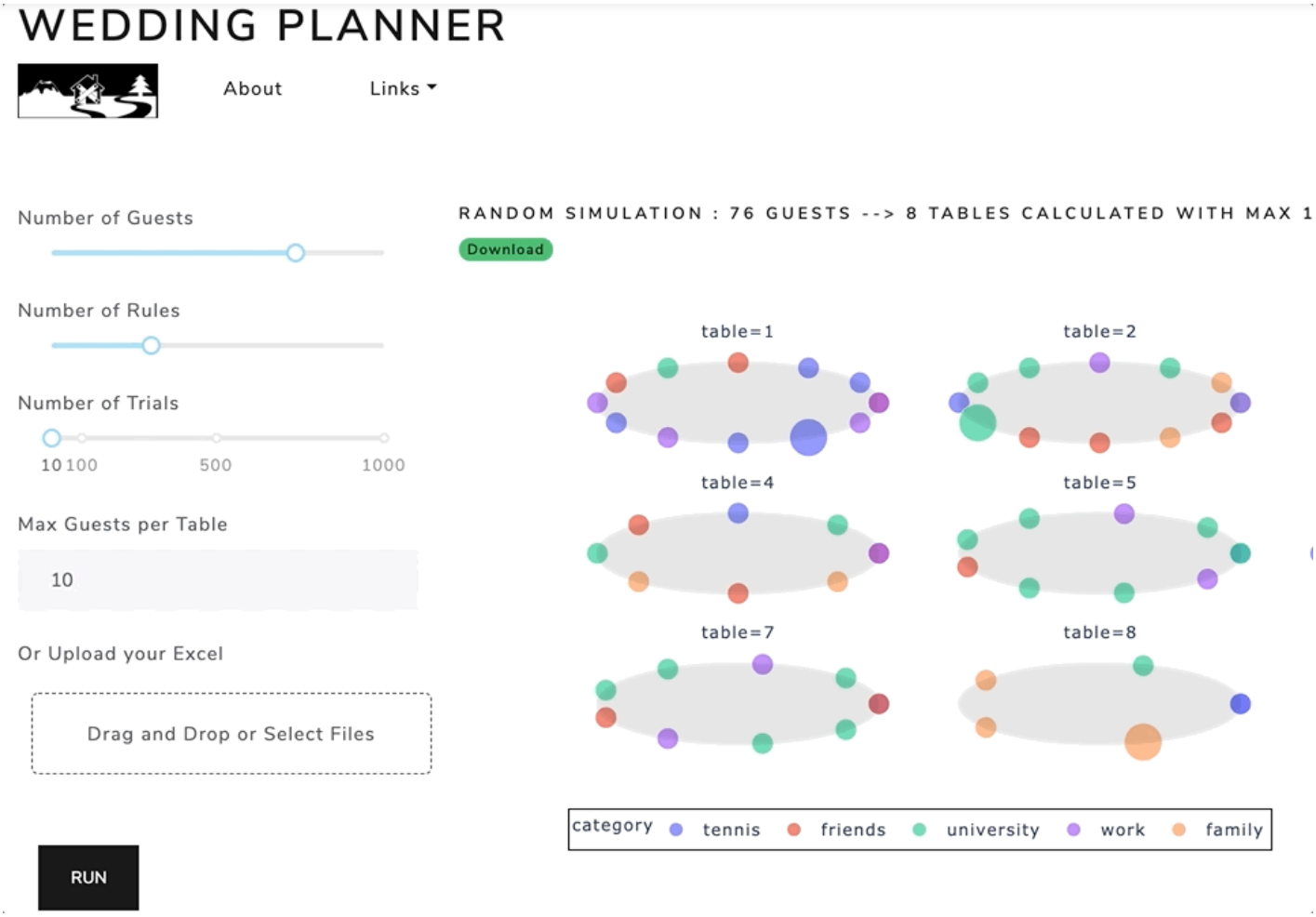


Image by Author

On the occasion of my best friend's wedding, I tried to make his life easier with a dashboard for seat arrangement. While coding it, I realized this simple project contains all the key features necessary to develop a basic but good prototype. So I thought it's worth sharing it.

Through this tutorial, I will explain how to build a complete Dash web application, using my Wedding Planner App as an example (link below).

Wedding Planner

Edit description

app-wedding-planner.herokuapp.com

I will present some useful Python code that can be easily applied in other similar cases (just copy, paste, run) and walk through every line of code with comments so that you can replicate this example (link to the full code below).

Get unlimited access Open in app

In particular, I will go through:

- Setup of the environment
- Data & Model
- Prepare the basic structure of the *dash* application (Navbar, Body, Layout)
- Inputs (Form, Slider, Manual, File Upload, Change inputs after an event)
- Outputs (*Plotly*, File Download, Spinner while loading)
- Deploy the app on *Heroku*

. . .

Setup

First of all, I will install the following libraries through the terminal:

```
## for application
dash==1.20.0
dash-bootstrap-components==0.12.2

## to produce random data
names==0.3.0

## for data processing
numpy==1.19.5
pandas==1.1.5

## for plotting
plotly==4.14.3

## for read and write excel files
openpyxl==3.0.7
xlrd==1.2.0
```

To make the dashboard look pretty, we'll use *Bootstrap*, a CSS/JS framework that contains design templates for forms, buttons, navigation, and other interface components. The package *Dash-Bootstrap-Components* enables easy integration of Bootstrap into our dash app.

The *names* package generates random names and I'll use it to create a dataset of random guests.

Please note that the latest version of *xlrd* (*2.0.0*) doesn't accept .xlsx files, therefore use the *1.2.0* if you want to upload Excel files.

Get unlimited access Open in app

of a random dataset.

```
import names
import pandas as pd
import numpy as np


'''
Generate random guests list
:parameter
    :param n: num - number of guests and length of dtf
    :param lst_categories: list - ["family", "friends", ...]
    :param n_rules: num - number of restrictions to apply (ex. if 1
                    then 2 guests can't be sit together)
:return
    dtf with guests
'''
def random_data(n=100, lst_categories=["family","friends",
                "work","university","tennis"], n_rules=0):
    ## basic list
    lst_dics = []
    for i in range(n):
        name = names.get_full_name()
        category = np.random.choice(lst_categories) if
                   len(lst_categories) > 0 else np.nan
        lst_dics.append({"id":i, "name":name, "category":category,
                         "avoid":np.nan})
    dtf = pd.DataFrame(lst_dics)

    ## add rules
    if n_rules > 0:
        for i in range(n_rules):
            choices = dtf[dtf["avoid"].isna()]["id"]
            ids = np.random.choice(choices, size=2)
            dtf["avoid"].iloc[ids[0]] = int(ids[1]) if
              int(ids[1]) != ids[0] else int(ids[1])+1

    return dtf
```

This function creates a table with guests' information.

Get unlimited access Open in app

Index	id	name	category	avoid
0	0	Ruth Caretto	work	nan
1	1	Harold Jaji	friends	9
2	2	Douglas Funderburke	friends	nan
3	3	Tamra Matt	university	6
4	4	Ivy Metzger	friends	nan
5	5	Clarence Julius	friends	4
6	6	Jason Bealer	work	nan
7	7	Gladys Vigne	family	nan
8	8	Rick Rener	university	0
9	9	Alberto Vasquez	tennis	10

Image by Author

I'll use the "category" column to display the guests with different colors:

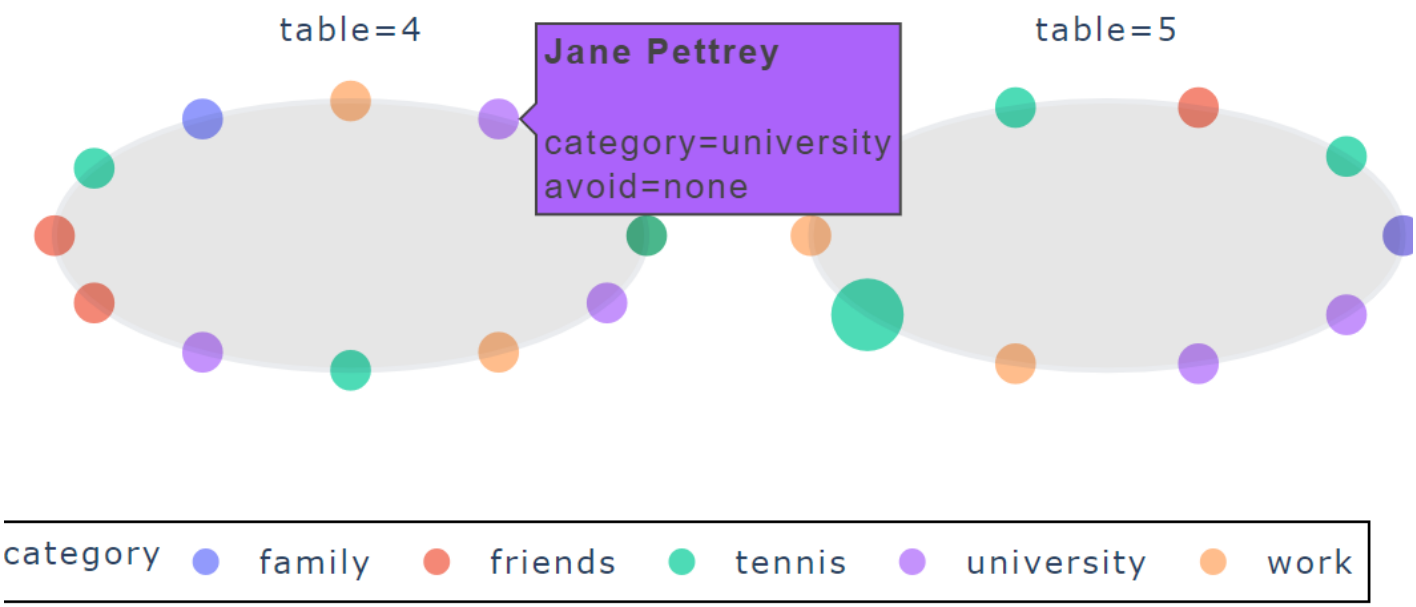


Image by Author

The "avoid" column will be used to make sure that two guests that hate each other will not be placed at the same

Get unlimited access Open in app

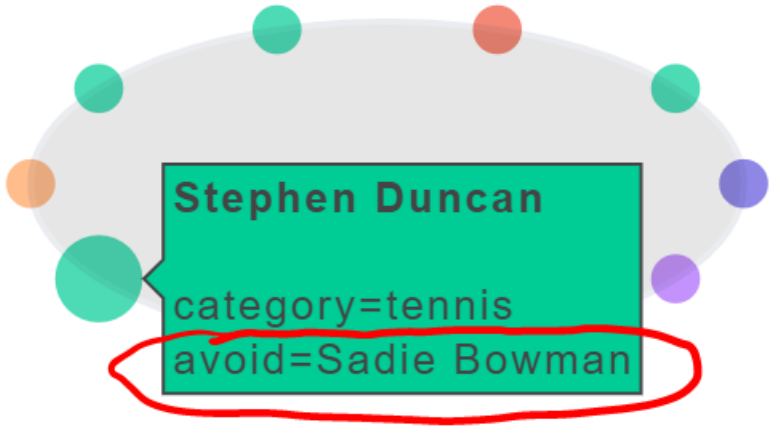


Image by Author

The app will be arranging seats based on:

- a reward of +1 when two people belonging to the same category are seated next to each other
- a penalty of -1 when two people that want to avoid each other are placed at the same table.

```
import numpy as np


class Model():

    def __init__(self, dtf, capacity=10, n_iter=10):
        self.n_tables = int(np.ceil( len(dtf)/capacity ))
        self.dic_tables = {i:capacity for i in range(self.n_tables)}
        self.dtf = dtf
        self.n_iter = n_iter


    @staticmethod
    def evaluate(dtf):
        score = 0
        for t in dtf["table"].unique():
            dtf_t = dtf[dtf["table"]==t]

            ## check penalties
            for i in dtf_t[~dtf_t["avoid"].isna()]["avoid"].values:
                if i in dtf_t["id"].values:
                    score -= 1

            ## check rewards
            seats = dtf_t["id"].values
            for n,i in enumerate(seats):
                cat = dtf_t[dtf_t["id"]==i]["category"].iloc[0]
```

Get unlimited access Open in app

```python
                prev_i = seats[n-1] if n > 0 else seats[-1]
                prev_cat = dtf_t[dtf_t["id"]==prev_i]["category"].iloc[0]
                if cat == prev_cat:
                    score += 1

        return score


    def run(self):
        best_dtf = self.dtf.copy()
        best_dtf["table"] = np.random.randint(low=1, high=self.n_tables+1, size=len(self.dtf))
        best_score = self.evaluate(best_dtf)

        for i in range(self.n_iter):
            self.dtf["table"] = np.random.randint(low=1, high=self.n_tables+1, size=len(self.dtf))
            score = self.evaluate(self.dtf)
            best_dtf = self.dtf if score > best_score else best_dtf

        return best_dtf
```

planner.py hosted with ❤ by GitHub view raw

This function returns the same dataframe with a new column for the table assigned:

Get unlimited access Open in app

id	name	category	avoid	table
0	Lottie Yardley	friends	9	4
1	Earl Morales	tennis	nan	2
2	Daniel Lekwa	tennis	0	2
3	Jennie Mcallister	tennis	nan	1
4	Josephine Dang	work	nan	4
5	Bruce Chase	work	3	2
6	Kevin Franklin	university	8	3
7	Mary Nelson	tennis	nan	1
8	Elsie Howell	work	nan	3
9	Maria Brown	friends	5	5

Image by Author

App Structure

Now we can start with the cool part: building the application. I'm gonna divide it into 3 sections: the Navbar, the Body, the Layout. Moreover, each section will contain 3 parts:

- Input — the section of the app where the user can insert and select the parameters and data that will be used by the back-end to return the desired output (The Navbar doesn't need Input).
- Output — the section of the app where the user can visualize the results.
- Callbacks — a decorator wrapping a function, in which you need to specify outputs, inputs, and states; the latter allow you to pass along extra values without firing the callbacks (Callbacks may look scary but actually they are your best friends).

```
# setup
import dash
from dash.dependencies import Input, Output, State
import dash_core_components as dcc
import dash_html_components as html
import dash_bootstrap_components as dbc

# App Instance
app = dash.Dash(name="name")
```

Get unlimited access Open in app

```
# Output
navbar = dbc.Nav()

# Callbacks
@app.callback()
def function():
    return 0

######################### Body #########################
# Input
inputs = dbc.FormGroup()

# Output
body = dbc.Row([
        ## input
        dbc.Col(md=3),
        ## output
        dbc.Col(md=9)
])

# Callbacks
@app.callback()
def function():
    return 0

######################### App Layout #########################
app.layout = dbc.Container(fluid=True, children=[
    html.H1("name", id="nav-pills"),
    navbar,
    html.Br(),html.Br(),html.Br(),
    body
])

######################### Run #########################
if __name__ == "__main__":
    app.run_server(debug=True)
```

Let's start with the **style.** The wonderful *Dash-Bootstrap-Components* offers a huge variety of predefined styles. You can check them out here. Once you choose one, you can insert it in the app instance as an external stylesheet. You can even use more than one:

```
theme = dbc.themes.LUX

css = 'https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css'

# App Instance
app = dash.Dash(name="name",
                external_stylesheets=[theme, css])
```

Let's move on to the top **Navbar**, I'll include a link, a popover, and a dropdown menu.

Get unlimited access Open in app

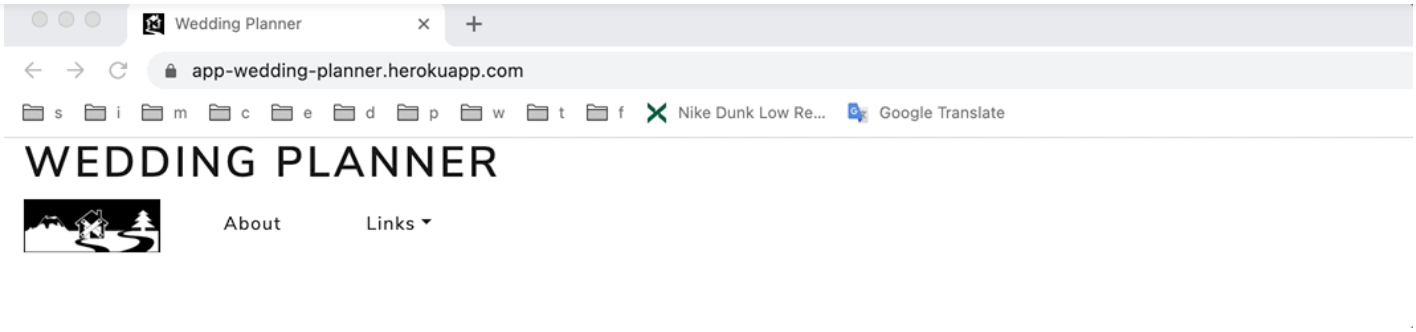


Image by Author

As you can see, the navbar is cool and reactive on click, with pop-ups and a drop-down menu. The “Links” drop-down menu is easy as you don’t need a Callback to make it work, while the “About” popover is a bit tricky.

Get unlimited access Open in app

```
# Output
navbar = dbc.Nav(className="nav nav-pills", children=[
    ## logo
    dbc.NavItem(html.Img(src="your/file/path", height="40px")),
    ## about
    dbc.NavItem(html.Div([
        dbc.NavLink("About", href="/", id="about-popover"
                    active=False),
        dbc.Popover(id="about", is_open=False,
                    target="about-popover", children=[
            dbc.PopoverHeader("How it works"),
            dbc.PopoverBody("some text...")
        ])
    ])),
    ## links
    dbc.DropdownMenu(label="Links", nav=True, children=[
        dbc.DropdownMenuItem([
          html.I(className="fa fa-linkedin"), "  Contacts"],
          href=config.contacts, target="_blank"),
        dbc.DropdownMenuItem([
          html.I(className="fa fa-github"), "  Code"],
          href=config.code, target="_blank")
    ])
])

# Callbacks
@app.callback(
output=[
 Output(component_id="about", component_property="is_open"),
 Output(component_id="about-popover", component_property="active")],
inputs=[
Input(component_id="about-popover", component_property="n_clicks")],
state=[
 State("about", "is_open"), State("about-popover", "active")])
def about_popover(n, is_open, active):
    if n:
        return not is_open, active
    return is_open, active
```

35

The navbar contains 3 nav-items: the logo, the “About” button, the drop-down menu. The “About” button

Get unlimited access Open in app

python function *about_popover()* expects 3 arguments because the Callback has one input and two states, and returns 2 variables because the Callback has two outputs. When the app starts and the button is not clicked *n=0*.

Please note that the drop-down menu (blue part) includes fonts imported with the external stylesheet (i.e. *className="fa fa-linkedin"*).

The navbar we've just seen is one of the elements of the final **Layout**, together with the title and the main Body:

```
######################### App Layout #########################
app.layout = dbc.Container(fluid=True, children=[
    html.H1("name", id="nav-pills"),
    navbar,
    html.Br(),html.Br(),html.Br(),
    body
])
```

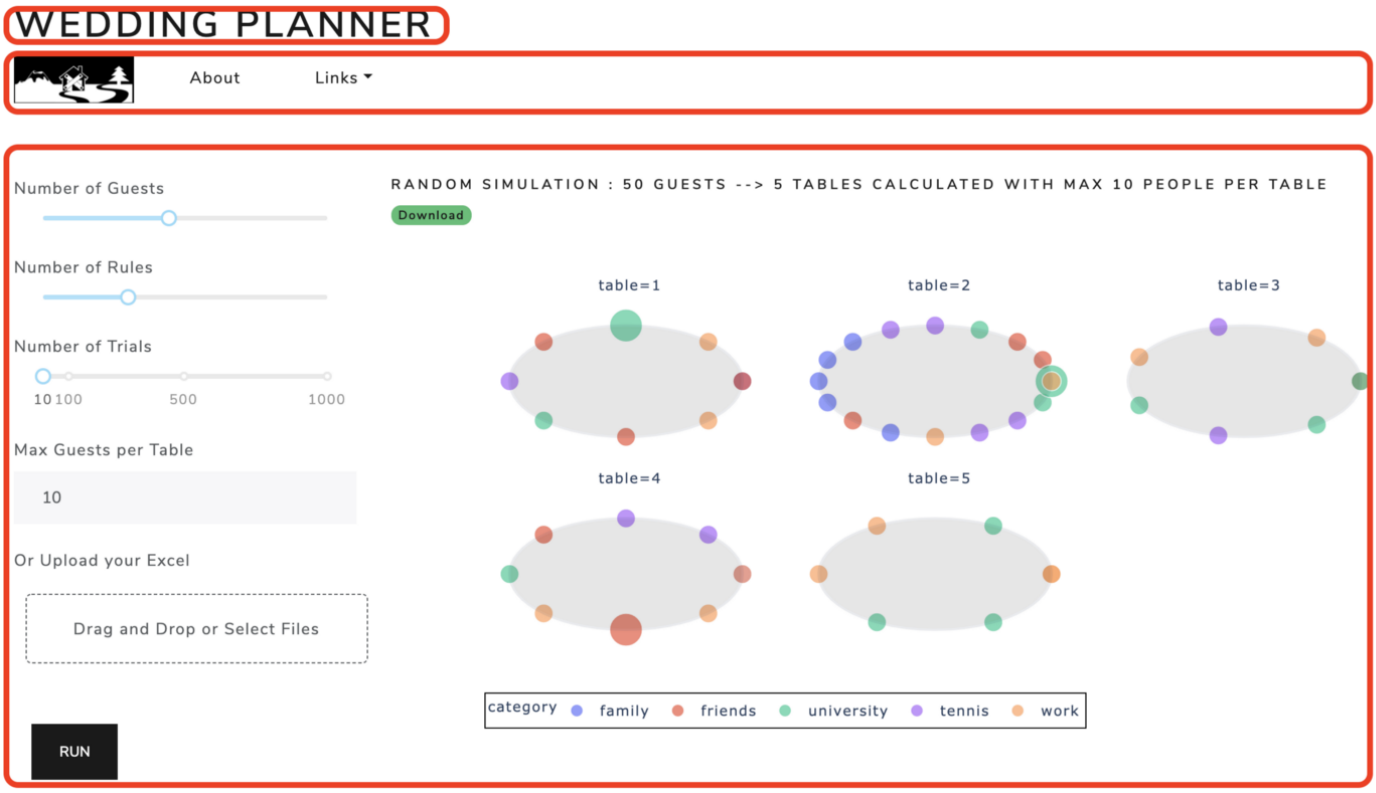


Image by Author

Now, let's talk about the elephant in the room… the **main Body**. It's composed of the Inputs (left side) and the Outputs (right side), which interact together thanks to Callbacks.

Get unlimited access Open in app

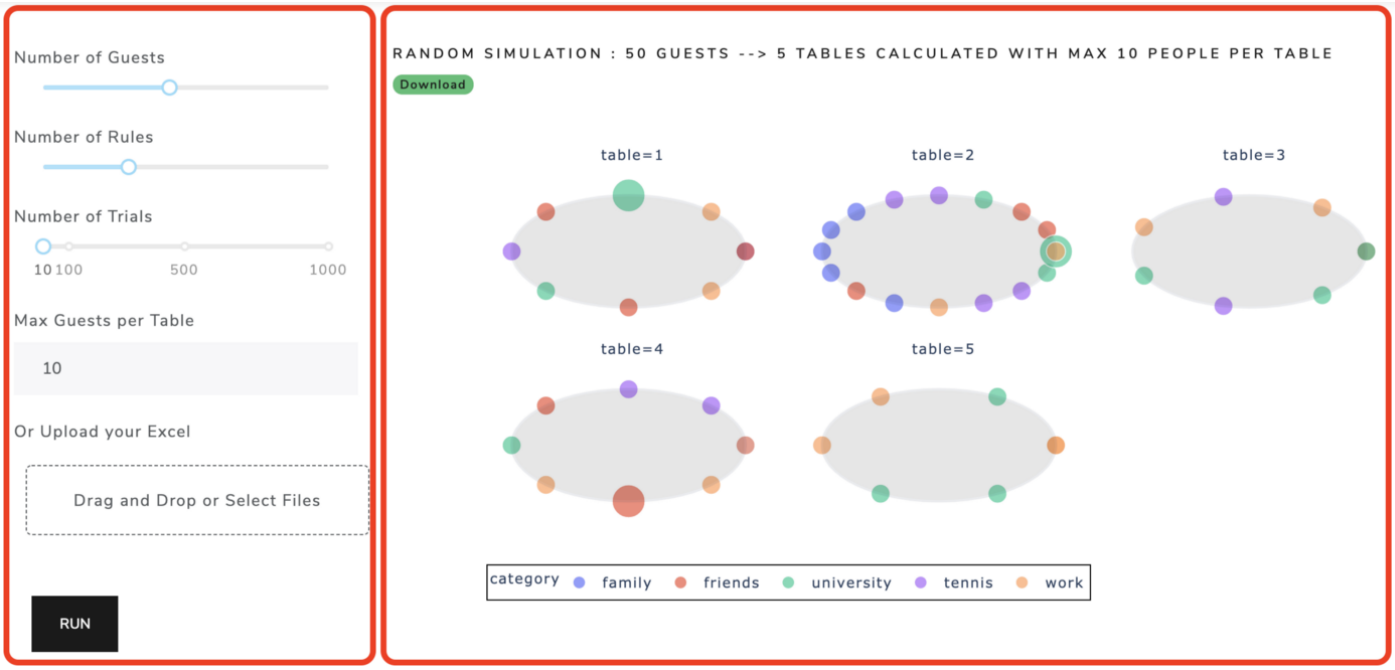
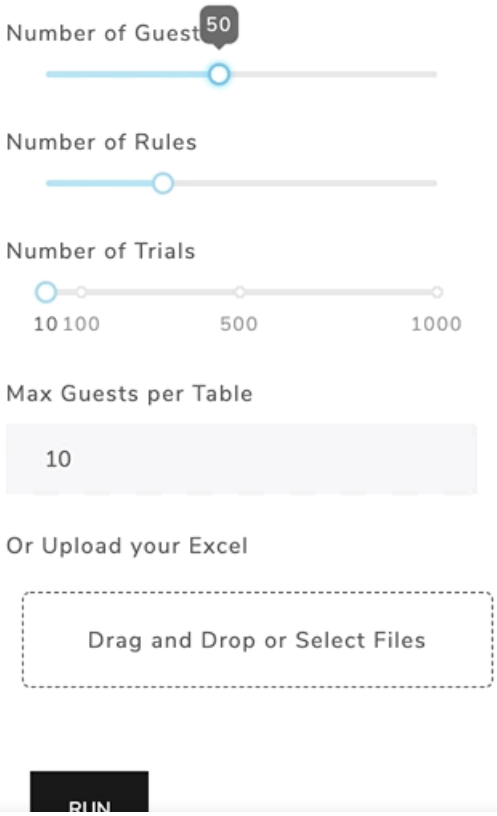


Image by Author

Inputs

Normally, inputs are wrapped in a Form group and are sent when the Form button is clicked. **Sliders and manual inputs** are the most common Form elements.



Get unlimited access Open in app

This is how you can create a normal slider:

```
dcc.Slider(id="n-guests", min=10, max=100, step=1, value=50,
           tooltip={'always_visible':False})
```

and here's how to force only specific values in the slider:

```
dcc.Slider(id="n-iter", min=10, max=1000, step=None,
           marks={10:"10", 100:"100", 500:"500", 1000:"1000"},
           value=0),
```

This is a simple manual input:

```
dbc.Input(id="max-capacity", placeholder="table capacity",
          type="number", value="10"),
```

Let's increase the difficulty and tackle the "**file Upload**" situation. I'm going to give users the possibility to upload an Excel file containing a similar dataset that we generated randomly:

	A	B	C	D
1	id	name	category	avoid
2	0	Marilyn Ric	friends	
3	1	Julie Hinerı	tennis	
4	2	Adam Marc	work	5
5	3	Steven Linz	family	
6	4	Michael Nie	university	
7	5	Charles Rob	work	
8	6	Shawn Wri	family	
9	7	Terry Shern	friends	
10	8	Tricia Ramc	family	
11	9	Amanda Gc	family	
12	10	Barbara De	friends	

Image by Author

When uploading a file, I want two things to happen:

Get unlimited access Open in app

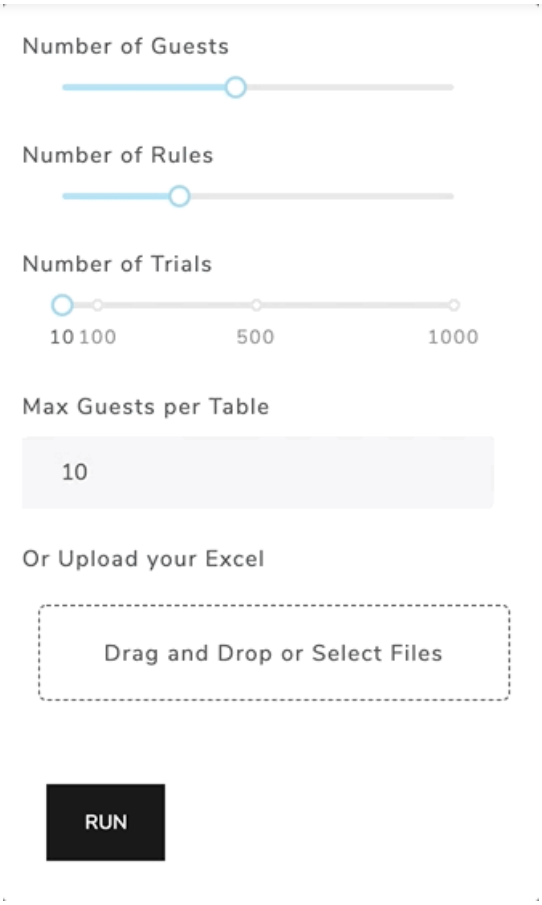


Image by Author

How do we achieve that? Easy, with a magic Callback that changes the CSS style of the HTML components:

Get unlimited access Open in app

```
## hide these 2 inputs if file is uploaded
html.Div(id='hide-seek', children=[

dbc.Label("Number of Guests", html_for="n-guests"),
        dcc.Slider(id="n-guests", min=10, max=100, step=1, value=50,
                   tooltip={'always_visible':False}),

dbc.Label("Number of Rules", html_for="n-rules"),
        dcc.Slider(id="n-rules", min=0, max=10, step=1, value=3,
                   tooltip={'always_visible':False})

], style={'display':'block'}),

## upload a file
dcc.Upload(id='upload-excel', children=html.Div([
               'Drag and Drop or ', html.A('Select Files')]),
               style={'width':'100%', 'height':'60px',
                      'lineHeight':'60px', 'borderWidth':'1px',
                      'borderStyle':'dashed',
                      'borderRadius':'5px', 'textAlign':'center',
                      'margin':'10px'} ),
html.Div(id='excel-name', style={"marginLeft":"20px"})

# Callbacks
@app.callback(
output=[
  Output(component_id="hide-seek", component_property="style"),
  Output(component_id="excel-name", component_property="children")],
inputs=[
  Input(component_id="upload-excel",component_property="filename")])
def upload_event(filename):
    div = "" if filename is None else "Use file "+filename
    return {'display':'block'} if filename is None else
           {'display':'none'}, div
```

In order to use the uploaded file, we need to parse it and transform it into a pandas dataframe, and we're going to use this function for that:

```
import pandas as pd
```

Get unlimited access Open in app

```
    :parameter
        :param contents: file
        :param filename: str
    :return
        pandas table
    '''
    def upload_file(contents, filename):
        content_type, content_string = contents.split(',')
        decoded = base64.b64decode(content_string)
        try:
            if 'csv' in filename:
                return pd.read_csv(io.StringIO(decoded.decode('utf-8')))
            elif 'xls' in filename:
                return pd.read_excel(io.BytesIO(decoded))
        except Exception as e:
            print("ERROR:", e)
            return 'There was an error processing this file.'
```

Before moving on with the Outputs, let's recap what we have seen so far. Here's the full code of the Inputs in the main Body:

```
########################## Body ##########################
# Input
inputs = dbc.FormGroup([
    ## hide these 2 inputs if file is uploaded
    html.Div(id='hide-seek', children=[

        dbc.Label("Number of Guests", html_for="n-guests"),
        dcc.Slider(id="n-guests", min=10, max=100, step=1, value=50,
                   tooltip={'always_visible':False}),

        dbc.Label("Number of Rules", html_for="n-rules"),
        dcc.Slider(id="n-rules", min=0, max=10, step=1, value=3,
                   tooltip={'always_visible':False})

    ], style={'display':'block'}),

    ## always visible
    dbc.Label("Number of Trials", html_for="n-iter"),
    dcc.Slider(id="n-iter", min=10, max=1000, step=None,
               marks={10:"10", 100:"100", 500:"500", 1000:"1000"},
               value=0),

    html.Br(),
    dbc.Label("Max Guests per Table", html_for="max-capacity"),
    dbc.Input(id="max-capacity", placeholder="table capacity",
              type="number", value="10"),

    ## upload a file
    html.Br(),
    dbc.Label("Or Upload your Excel", html_for="upload-excel"),
    dcc.Upload(id='upload-excel', children=html.Div([
               'Drag and Drop or ', html.A('Select Files')]),
               style={'width':'100%', 'height':'60px',
```

Get unlimited access Open in app

```
    ## run button
    html.Br(),html.Br(),
    dbc.Col(dbc.Button("run", id="run", color="primary"))
])


# Callbacks
@app.callback(
 output=[
  Output(component_id="hide-seek", component_property="style"),
  Output(component_id="excel-name", component_property="children")],
 inputs=[
  Input(component_id="upload-excel",component_property="filename")])
def upload_event(filename):
    div = "" if filename is None else "Use file "+filename
    return {'display':'block'} if filename is None else
           {'display':'none'}, div
```

Outputs

The back-end shall produce 3 outputs: the title, a link to download the results as an Excel file, and obviously the plot.

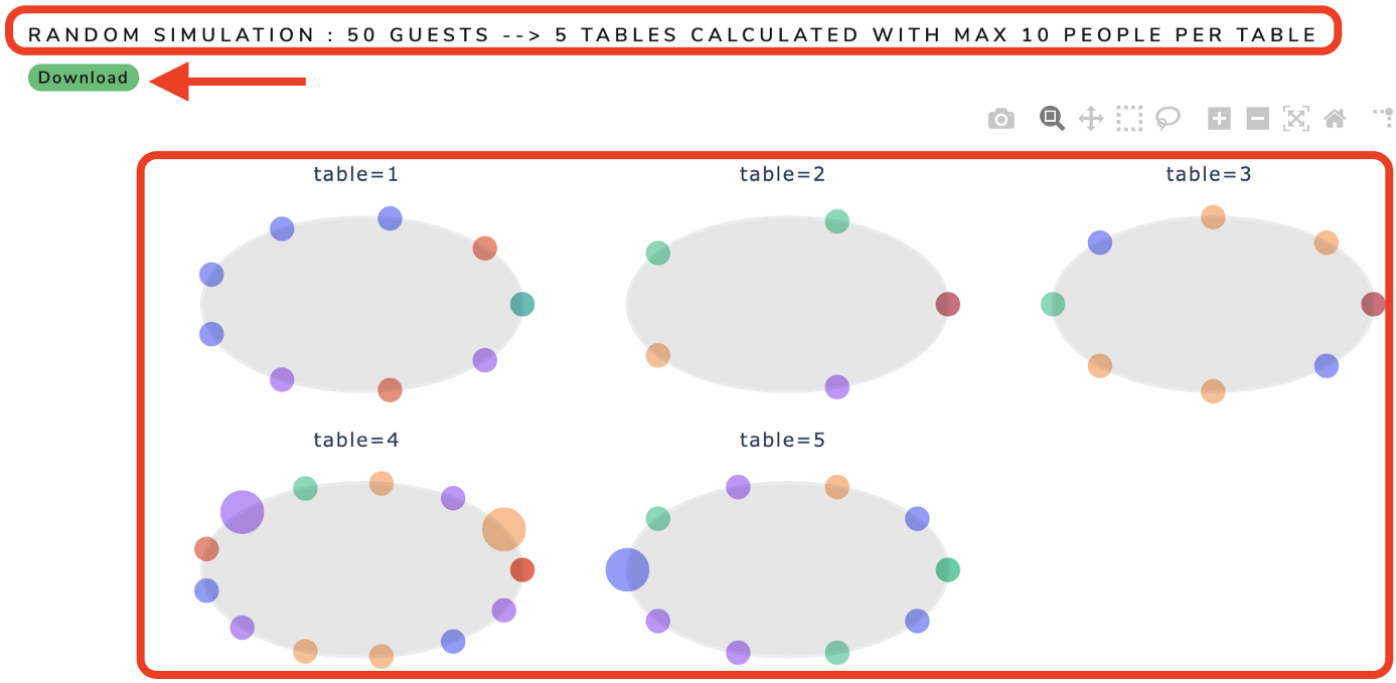


Image by Author

Let's get started with the plot made with **Plotly.** Essentially, there are two major modules of this amazing graphic library: *plotly express* and *graph_objects*. The former is a high-level graphic tool containing functions that can create entire figures at once (I find it similar to *seaborn*), while the latter allows you to build a figure brick by brick (it is in fact what *plotly express* runs under the hood).

Get unlimited access Open in app

So… what's this?

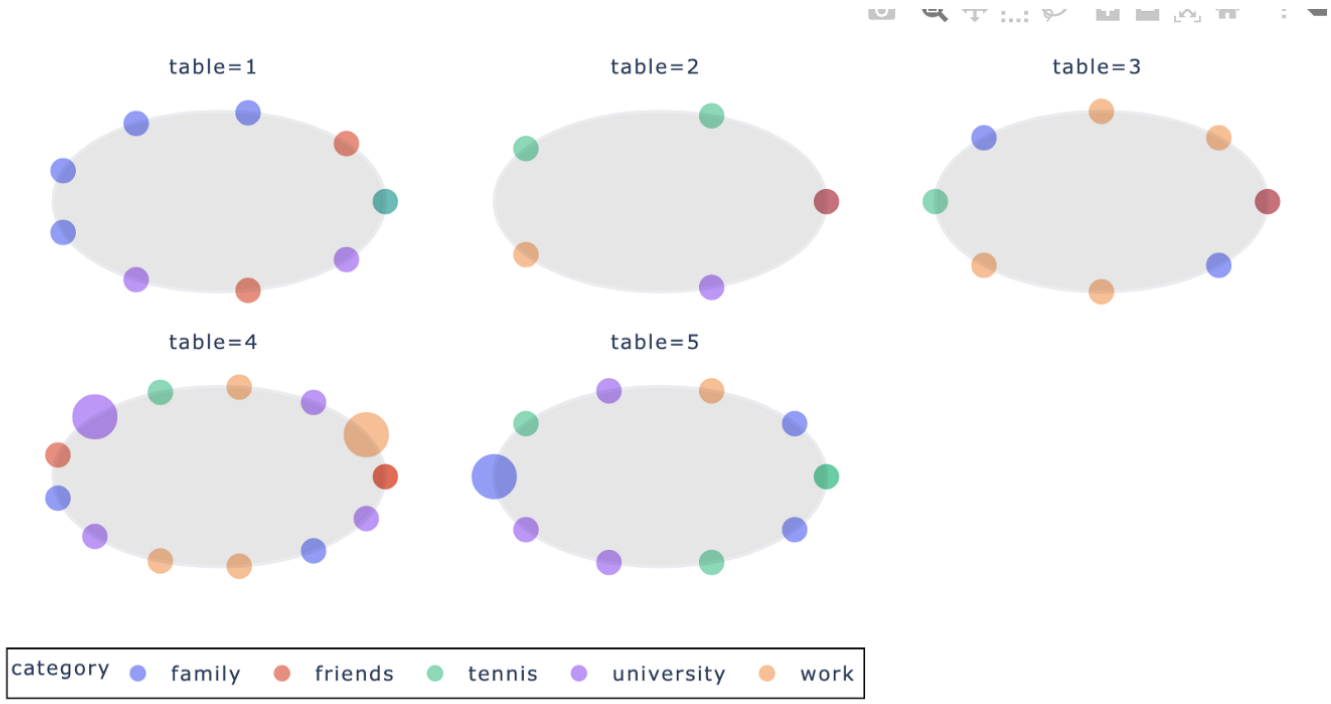


Image by Author

It may look like a drawing but it is a proper geometric figure: it's a scatter plot in which the color is based on the category guests belong to (family, friends, …), the size is determined by whether a guest wants to avoid someone or not (basically I'm highlighting the problematic ones), and the facet component is linked to the table assigned to each guest. Facet plots are figures made up of multiple subplots which have the same set of axes, where each subplot shows a subset of the data.

First of all, I need to add the x and y coordinates for the plot using the circle equation: $(x, y) = (r*\cos\theta, r*\sin\theta)$. Then I add the "size" column based on the "avoid" column:

```
def prepare_data(dtf):
    ## mark the rules
    dtf["avoid"] = dtf["avoid"].apply(lambda x:
                    dtf[dtf["id"]==x]["name"].iloc[0] if
                    pd.notnull(x) else "none")
    dtf["size"] = dtf["avoid"].apply(lambda x: 1 if x == "none"
                    else 3)

    ## create axis
    dtf_out = pd.DataFrame()
    lst_tables = []
    for t in dtf["table"].unique():
        dtf_t = dtf[dtf["table"]==t]
        n = len(dtf_t)
```

Get unlimited access Open in app

```
    return dtf_out.reset_index(drop=True).sort_values("table")
```

Index	category	avoid	table	size	x	y
5	nis	none	1	1	1	0
6	nis	none	1	1	1	-2.44929e-16
2	nis	none	2	1	1	0
3	nis	Lottie Yardley	2	3	-1	1.22465e-16
4	k	Jennie Mcallister	2	3	1	-2.44929e-16
7	versity	Elsie Howell	3	3	1	0
8	k	none	3	1	1	-2.44929e-16
0	ends	Maria Brown	4	3	1	0
1	k	none	4	1	1	-2.44929e-16
9	ends	Bruce Chase	5	3	1	0

Image by Author

Then I can simply use plotly commands to produce figures and specify what information visualize when the mouse hovers over the points:

```
import plotly.express as px

fig = px.scatter(dtf, x="x", y="y", color="category",
                 hover_name="name", facet_col="table",
                 facet_col_wrap=3, size="size",
                 hover_data={"x":False, "y":False, "category":True,
                             "avoid":True, "size":False,
                             "table":False}
              )

fig.add_shape(type="circle", opacity=0.1, fillcolor="black",
              col="all", row="all", exclude_empty_subplots=True,
              x0=dtf["x"].min(), y0=dtf["y"].min(),
              x1=dtf["x"].max(), y1=dtf["y"].max()
              )

fig.update_layout(plot_bgcolor='white',
                  legend={"bordercolor":"black",
                          "borderwidth":1,
                          "orientation":"h"}
              )
```

Get unlimited access Open in app

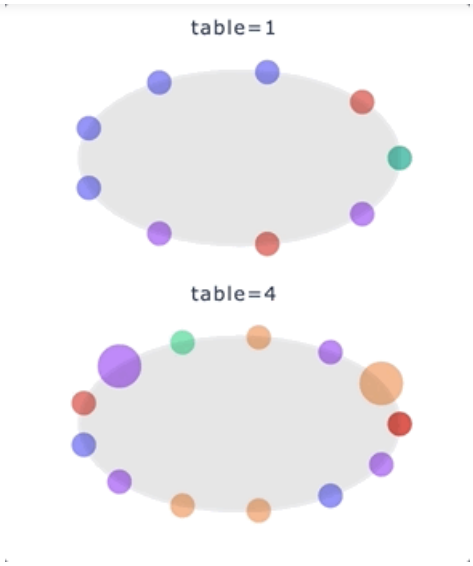


Image by Author

Full code for plotly (and the title):

Get unlimited access Open in app

Now that the plot is done, how to **download the results as an Excel file**? We just need a function that transforms the pandas dataframe into a file and passes the link to download it to the UI:

```
import pandas as pd
import io
import base64

'''
Write excel
:parameter
    :param dtf: pandas table
:return
    link
'''
def download_file(dtf):
    xlsx_io = io.BytesIO()
    writer = pd.ExcelWriter(xlsx_io)
    dtf.to_excel(writer, index=False)
    writer.save()
    xlsx_io.seek(0)
    media_type = 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
    data = base64.b64encode(xlsx_io.read()).decode("utf-8")
    link = f'data:{media_type}; base64,{data}'
    return link
```

On the front-end side, we have to add the HTML link for downloading and do the usual trick with a Callback:

```
# Output
body = dbc.Row([
        ## input
        dbc.Col(md=3, children=[
            inputs,
            html.Br(),html.Br(),html.Br(),
        ]),
        ## output
        dbc.Col(md=9, children=[
            dbc.Spinner([
                ### title
                html.H6(id="title"),
                ### download
```

Get unlimited access Open in app

```
                dcc.Graph(id="plot")
            ], color="primary", type="grow"),
        ])
])

# Callbacks
@app.callback(
output=[
 Output(component_id="title", component_property="children"),
 Output(component_id="plot", component_property="figure"),
 Output(component_id='download-excel', component_property='href')],
inputs=[
 Input(component_id="run", component_property="n_clicks")],
state=[
  State("n-guests","value"),
  State("n-iter","value"),
  State("max-capacity","value"),
  State("n-rules","value"),
  State("upload-excel","contents"),
  State("upload-excel","filename")])
def results(n_clicks, n_guests, n_iter, max_capacity, n_rules,
            contents, filename):
    if contents is not None:
        dtf = upload_file(contents, filename)
    else:
        dtf = random_data(n=n_guests, n_rules=n_rules)
    dtf = Model(dtf, float(max_capacity), int(n_iter)).run()
    out = Plot(dtf)
    return out.print_title(max_capacity, filename),
           out.plot(),
           download_file(dtf.drop("size", axis=1))
```

As you may have noticed, the outputs (title, download, plot) are wrapped inside a **Spinner**, which renders this nice effect of loading state while elaborating the inputs:

Get unlimited access Open in app

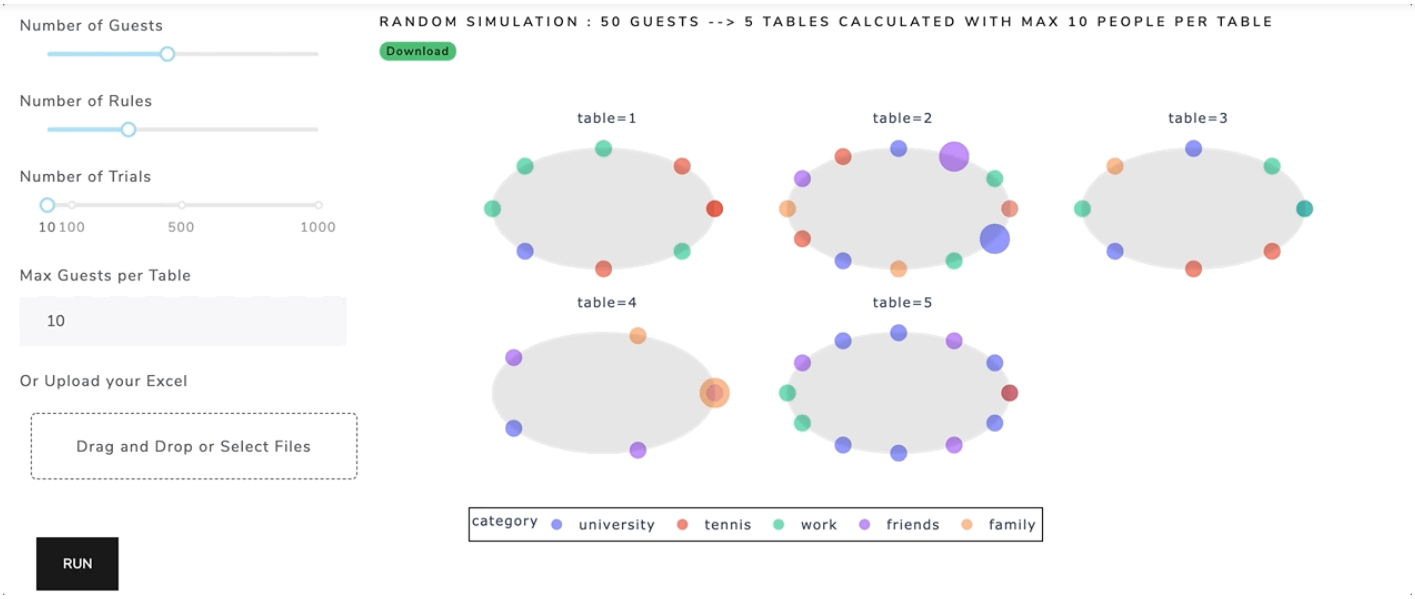


Image by Author

Deploy

Finally, we are ready to deploy this application. Here's the full code of the dash app (you can check out the rest of the repo on GitHub):

Get unlimited access Open in app

Personally, I like **Heroku** for deploying prototypes**.** You're gonna need to add a *requirements.txt* and a *Procfile*. If you need help with that, you can find detailed tutorials here and here.

Conclusion

This article has been an (almost) complete tutorial about how to **build a nice web application with Python Dash.** This app is pretty straightforward as it doesn't have any DB and user login feature (maybe material for the next tutorial?). Here I just wanted to demonstrate how you can easily transform your ideas into a prototype to show the world. And now that you know how it works, you can develop your own app.

I hope you enjoyed it! Feel free to contact me for questions and feedback or just to share your interesting projects.

👉 Let's Connect 👈

. . .

*This article is part of the series **App Development with Python**, see also:*

How to embed Bootstrap CSS & JS in your Python Dash app

Build a Covid-19 infection forecaster app with Dash Bootstrap Components

towardsdatascience.com

Build & Deploy a Telegram Bot with short-term and long-term memory

Create a Chatbot from scratch that remembers and reminds events with Python

pub.towardsai.net

Get unlimited access Open in app

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

Get this newsletter

Emails will be sent to chris.a.lindgren@gmail.com.
Not you?